Multiple system-wide low-power states

Kevin Hilman (BayLibre), Ulf Hansson (Linaro)



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Part 1:
s2idle state selection



Background & Context

Modern SoCs have multiple system-wide low-power states.

Low-power idle (CPUidle) chooses between the many states.

System-wide suspend-to-idle (s2idle) always picks the deepest state.



The Problem Statement

Just like low-power idle (CPUidle) can chose a state based on `$CRITERIA`

We want to system-wide suspend-to-idle (s2idle) to choose between different low-power states

But… How to decide which state to pick?



Today: new user-space interface (Nov '25)

Existing user-space interface: `/dev/cpu_dma_latency`

Used by low-power idle idle-state selection (CPUidle)

New user-space interface: `/dev/cpu_wakeup_latency`

Used by system-wide low-power state selection (s2idle)
AND low-power idle (CPUidle)
AND PM domain-idle-state selection

Ulf's series (v4): Queued for v6.19

https://lore.kernel.org/all/20250716123323.65441-1-ulf.hansson@linaro.org/



Good, but…

Devices may also have per-device QoS constraints (`dev_pm_qos*`)

PM domains should respect these constraints for all devices within domain

Today: constraints are respected for runtime PM decisions

but NOT system-wide suspend (to idle) decisions.



Proposal: per-device QoS for s2idle

Re-use existing per-device QoS latency

```
# echo 500000 > /sys/devices/platform/.../power/pm_qos_resume_latency_us
```

AND… set flag to indicate that this constraint ALSO applies to system-wide suspend

```
# echo 1 > /sys/devices/platform/.../power/pm_qos_latency_sys
```

If flag not set, constraint only applies to runtime PM decisions.

Kevin's proposa/RFCI:



https://lore.kernel.org/r/7hh5uwkutu.fsf@baylibre.com

Part 2:
PM domain hierarchy



The problem: Describing PM domain hierarchy

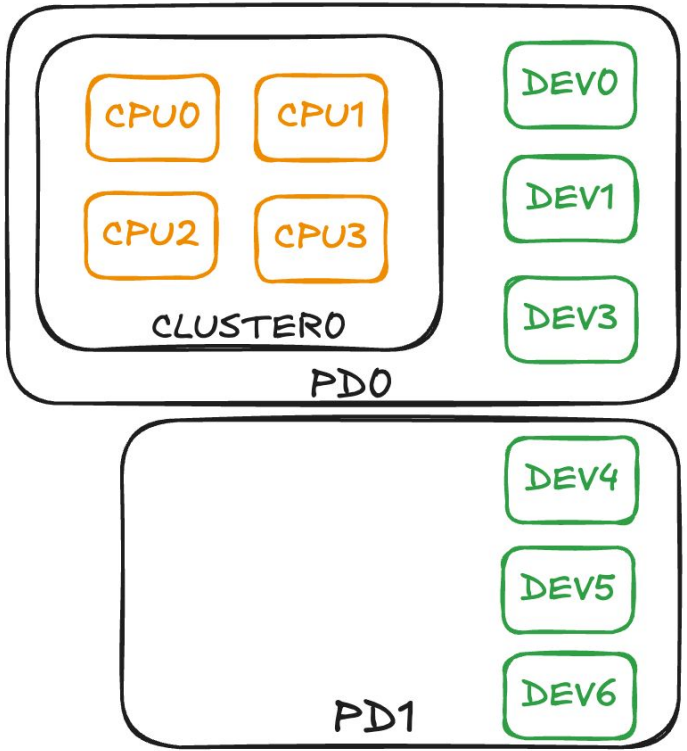
CPUs/clusters may be in PM domains with other (non-CPU) devices

Today: PM domains support hierarchy, but only for "simple" (#power-domain-cells = 0)

Child domains of same type (e.g. SCMI) may be in different domains

e.g. `DEV0` in `PD0`; `DEV4` in `PD1`

Also…. For s2idle state selection, any constraints in the CPU hierarchy should be respected.



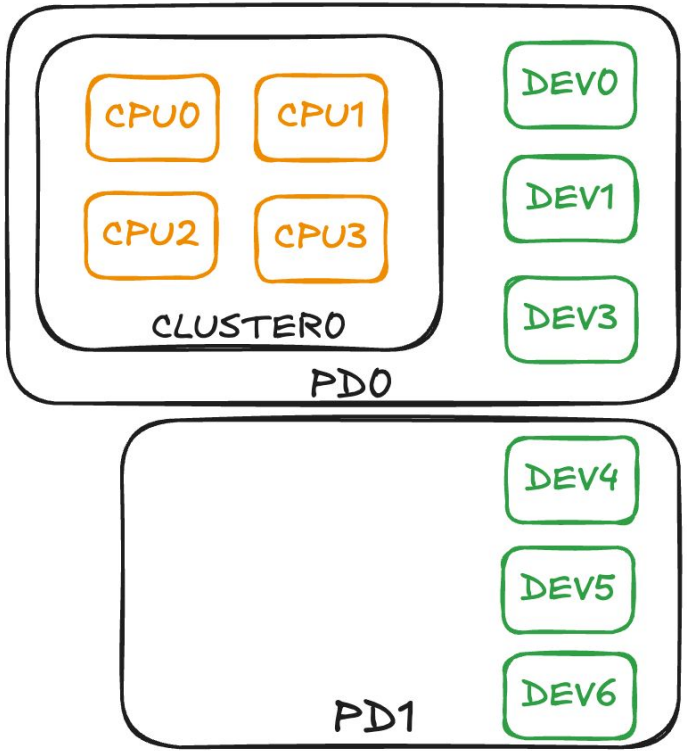
Suggestion: use nexus node map

```
scmi_pds: protocol@11 {
    #power-domain-cells = <1>;
    power-domain-map = <15 &PD0>,  /* DEV0 */
                       <...>,
                       <19 &PD1>;  /* DEV4 */
};

PD0: power-controller-0 {
    #power-domain-cells = <0>;
    domain-idle-states = <...>;
};

CLUSTER0_PD: power-controller-cluster0 {
    #power-domain-cells = <0>;
    domain-idle-states = <...>;
    power-domains = <&PD0>;
};

cpu0: cpu@0 {
    power-domains = <&CLUSTER0_PD>;
    power-domain-names = “psci”;
};
```



Needed:

Iterator for nexus-node maps

Recently contributed
- iterator for interrupt-map
- Not usable for generic maps

RFC posted. Help wanted.




```
scmi: scmi {
        compatible = "arm,scmi-smc";

        scmi_pds: protocol@11 {

                power-domain-map = <15 &MAIN_PD>, /* TIMER0 */
                                   <16 &MAIN_PD>, /* TIMER1 */
                                   <17 &MAIN_PD>, /* TIMER2 */
                                   <18 &MAIN_PD>, /* TIMER3 */
                                   <19 &WKUP_PD>, /* WKUP_TIMER0 */
                                   <20 &WKUP_PD>, /* WKUP_TIMER1 */
                                   <26 &MAIN_PD>, /* MMCSD1 */
                                   <27 &MAIN_PD>, /* MMCSD2 */
                                   <28 &MAIN_PD>, /* MMCSD0 */
                                   <34 &MAIN_PD>, /* GPIO0 */
                                   <35 &MAIN_PD>, /* GPIO2 */
                                   <36 &WKUP_PD>, /* WKUP_GPIO0 */
                                   <53 &MAIN_PD>, /* I2C0 */
                                   <54 &MAIN_PD>, /* I2C1 */
                                   <55 &MAIN_PD>, /* I2C2 */
                                   <56 &MAIN_PD>, /* I2C3 */
                                   <57 &WKUP_PD>; /* WKUP_I2C0 */
```

Kevin's RFCs

pmdomain v4:
https://lore.kernel.org/r/20251119-pmdomain-hierarchy-onecell-v4-1-f25a1d5022f8@baylibre.com

OF parser/iterator:
https://lore.kernel.org/r/20251119-topic-lpm-of-map-iterator-v6-18-v1-1-1f0075d771a3@baylibre.com



Part 3: s2idle only states



Problem statement

Is there a “right way” to add an idle-state (or domain-idle-state) that is only available for s2idle? e.g. it should not be selected by CPUidle

Can just use really high enter/exit/residency latencies?

Ugly, but does work.



Thank you



